VNUHCM - UNIVERSITY OF SCIENCE

REPORT

SHARED L2 CACHE DESIGN SPACE EXPLORATION USING GEM5

Category: System Verification & Performance Analysis

Author: Huynh Vu Thuc Quyen

Ho Chi Minh City, October 29, 2026

I. WEEKLY HIGHLIGHTS

This week, I have completed the following tasks:

- **Baseline Preparation:** Successfully configured two cache versions on gem5 for comparative analysis: The WITHOUT MSHR version (Blocking Cache - Original Baseline) and the WITH MSHR version (Non-blocking Cache).
- **Workload Execution:** Executed the `mbench.c` benchmark (Mode 1: Capacity-bound - inducing LRU thrashing to overflow the L2 cache) on both configurations to gather empirical data, serving as the basis for trade-off analysis and design decision-making.

II. WORKLOAD PERFORMANCE RESULTS ACROSS CONFIGURATIONS

The `mbench.c` workload (Mode 1) was executed concurrently on both configuration versions. Results extracted from the `stats.txt` log reveal a distinct performance divergence under heavy cache miss traffic:

Metric	WITHOUT MSHR (Baseline)	WITH MSHR
Total Cycles	~3,500,000 cycles	~1,500,000 cycles
Speedup	1.0x	2.33x
Average IPC	Severely degraded (< 0.15)	Maintained stably (~ 0.35)
Pipeline State	Continuously stalled	Pipeline flows continuously via **Hit-under-Miss**

III. OPERATIONAL MECHANISM COMPARISON TABLE

Building on the empirical results in Section II, the table below analyzes the microarchitectural mechanisms driving the performance differences between the two versions:

Criteria	WITHOUT MSHR	WITH MSHR
Miss Handling Mechanism	Upon a **Cache Miss**, the cache halts all operations, locks the interface bus, and forces the CPU to stall until data is returned from RAM.	Stores the missed instruction context into an **MSHR Entry** and immediately releases the interface bus.
Concurrency	**Low:** Processes only a single memory transaction at a time.	**Very High:** Allows the CPU to continue executing independent instructions (*Hit-under-Miss, Miss-under-Miss*).
Bus Bandwidth	Issues a separate request to RAM for every miss. Prone to bus congestion under burst access.	Supports **MSHR Coalescing** (merging multiple misses to the same 64B Cache Line into a single request).
Hardware & Area	**Minimal:** Zero die area overhead, highly power efficient.	**Complex:** Incurs area overhead for the MSHR array; risk of **Back Pressure** when MSHRs are full.

IV. EVALUATION: WHICH CONFIGURATION SUITS CVA6?

Architectural Conclusion

The CVA6 processor is best suited for and requires the WITH MSHR configuration (Non-blocking Cache).

Microarchitectural Rationale:

- CVA6 is a 64-bit application processor core featuring a **6-stage pipeline** and a **Load-Store Unit (LSU)** designed for operating systems like Linux.
- Based on the metrics in Section II, if CVA6 uses the **Without MSHR** version, every L1 Cache Miss causes all 6 pipeline stages of CVA6 to suffer a prolonged **Pipeline Stall** for hundreds of clock cycles while waiting for RAM (consuming up to 3.5 million cycles). This severely degrades IPC.
- Conversely, with the **With MSHR** version, CVA6 can dispatch the miss request into an MSHR, freeing the LSU to continue executing subsequent independent instructions within the pipeline, thereby reducing execution time to just 1.5 million cycles.

Where is the WITHOUT MSHR version appropriate?

- **Suitable for simple microcontroller cores (MCUs):** Since instruction fetch flows in low-end microcontrollers are typically sequential and tightly dependent, pausing the pipeline upon a new instruction fetch is a natural behavior to minimize die area. However, this architecture is a performance bottleneck and entirely unusable for high-performance systems like CVA6.